

SaaS-Bench: 计算机使用智能体能否利用真实 SaaS 完成专业工作流?

Kean Shi^{1,2,*}, Zihang Li^{2,*}, Tianyi Ma^{1,*}, Zengji Tu^{1,2}, Jialong Wu^{1,2}, Wendong Xu¹,
Xinbo Xu^{1,2}, Qingyao Yang^{1,3}, Ruoyu Wu^{1,2}, Weichu Xie², Ming Wu⁴, Jason Zeng⁴,
Michael Heinrich⁴, Elvis Zhang⁵, Liang Chen^{1,†}, Kuan Li^{1,†}, Baobao Chang^{2,†}

¹UniPat AI, ² 北京大学, ³ 香港大学, ⁴OG Labs, ⁵Pipeline Lab

摘要

计算机使用智能体 (Computer-Using Agents, CUA) 正在迅速把大语言模型 (LLM) 的能力从基于文本的推理扩展到在网页浏览器、图形用户界面 (GUI) 等更复杂环境中执行动作。然而, 现有网页与 GUI 智能体基准通常依赖简化环境、孤立任务或短程交互, 因而难以评估智能体在真实专业工作流中的能力。软件即服务 (SaaS) 环境天然适合作为 CUA 的评测载体: 现代数字化工作有很大一部分在其中完成, 而这些环境本身就涉及动态系统状态、跨应用协同、领域专门知识和长程依赖。为此, 我们提出 **SaaS-Bench**: 该基准建立在六个专业领域的 23 个可部署 SaaS 系统之上, 包含 106 个源自真实工作场景的任务。这些任务要求长程执行, 同时覆盖纯文本与多模态设置, 并通过带权验证检查点评估严格任务完成情况与部分进展。实验表明, 具有代表性的 LLM 智能体在 SaaS-Bench 上表现困难; 即便最强模型, 端到端完成的任务也不足 4%, 暴露出其在规划、状态跟踪、跨应用上下文维持和错误恢复方面的局限。复现代码见 [UniPat-AI/SaaS-Bench](#)。

1 引言

大语言模型 (LLM) 的近期进展催生了计算机使用智能体 (CUA) (Qin et al. (2025); Wang et al. (2025); OpenAI (2025); Anthropic (2024), 这标志着范式从被动理解转向主动执行 (Zhou et al. (2023); Xie et al. (2024); He et al. (2024)。传统模型只关注语言理解与生成, 而 CUA 能够在图形用户界面、网页浏览器和 API 等多种界面中采取动作, 从而与真实软件系统交互。这使它们能够完成信息检索、数据操作和多步任务执行等端到端工作流。因此, 如何评估 CUA 的真实能力已成为核心问题。

然而, 现有基准无法准确反映智能体的现实能力, 导

致性能被系统性高估 (Deng et al. (2023); Zhou et al. (2023); Koh et al. (2024); Xie et al. (2024)。第一, 它们在应用层面的复杂度有限。即使某些基准采用可执行或可自行托管的网页环境, 其页面逻辑、后端约束和状态转移通常仍比真实 SaaS 系统简单。第二, 它们没有捕捉真实专业工作流: 任务通常局限于目标经过简化的单个孤立应用场景, 而实际专业工作天然涉及多个系统的协同、领域专门知识和结构化的多步流程。第三, 它们缺少真实 SaaS 工作流所需的长程依赖: 完成一个任务可能需要 100 次以上交互。因此, 现有基准难以说明 CUA 能否在真实且高价值的场景中有效工作。

为弥补这些局限, 必须在能够反映现实工作的环境中评估智能体。软件即服务 (SaaS) 平台已经成为现代知识工作的主要基础设施, 并广泛用于客户关系管理、财务、运营和客户支持等领域 (China (2025); Gartner (2024)。这些系统天然具有三项关键属性: (1) 复杂且真实的环境, 具备完整的前后端交互与动态状态依赖; (2) 具有经济意义的工作流, 涉及结构化多步流程与跨系统协调; (3) 内在的长程任务结构, 需要跨越多个阶段持续交互。与合成或简化基准不同, SaaS 系统面向真实用户和真实运营流程而设计, 其中动作与持久化数据及系统约束紧密耦合。因此, 智能体在此类环境中的行为能够更忠实地反映其实用性与稳健性。就真实性、复杂度与实际相关性而言, SaaS 平台由此成为评估 CUA 的理想试验场。

基于这一认识, 我们提出 SaaS-Bench, 如图 2 所示, 该基准用于在真实 SaaS 环境下评估 CUA。SaaS-Bench 的设计遵循三项原则。第一, 它提供真实且可部署的 SaaS 环境: 这些环境由现实中的开源 SaaS 系统构建, 保留完整的前后端逻辑和动态约束, 同时支持本地部署。该设计既迫使智能体在真实系统动态下工作、无法利用简化环境中的捷径, 又为系统评测保留了可复现性和可控性。第二, 它纳入真实的组合式工作流, 模拟跨应用协调和多模态任务要求。这些工作流反映真实系统中的典型使用模式, 要求智能体整合异构信息并协调多个子系统。第三, 它引入平均交互超过 100 步的长程任务, 明确评估

* 共同核心贡献者。

† 通讯作者: Liang Chen <liangchen@unipat.ai>, Kuan Li <kuanli@unipat.ai>, Baobao Chang <chbb@pku.edu.cn>。

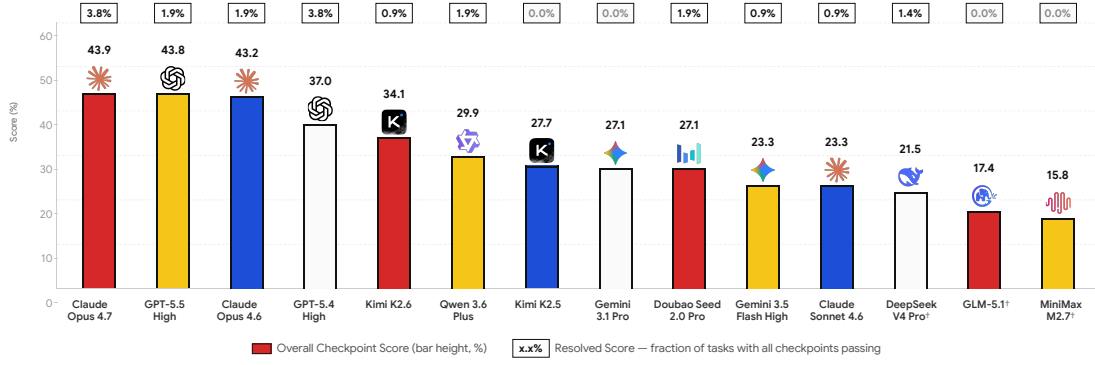


图 1: SaaS-Bench 排行榜。图中给出 14 个前沿模型在 106 个长程 SaaS 任务上的总体检查点分数（柱高）与完全解决分数。[†] 表示纯文本模型。

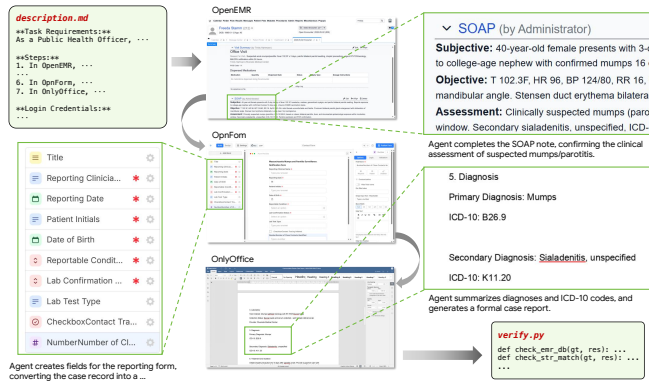


图 2: SaaS-Bench 为在可部署 SaaS 环境中评估 CUA 提供了一个真实基准。它由分属六个专业领域的 23 个真实 SaaS 系统构成，支持 106 个反映现实 SaaS 工作流的任务。

规划、状态管理和错误恢复能力。任务深度与依赖关系的大幅增加揭示了短程设置中常被掩盖的失效模式，从而更全面地评估智能体行为。表 1 从若干关键维度概括了 SaaS-Bench 与现有网页及 GUI 智能体基准的差异。

我们的贡献概括如下：

- **真实且可部署的 SaaS 基准环境。**我们在六个专业领域的 23 个真实 SaaS 系统上构建 SaaS-Bench，保留前后端动态，并可通过 Docker 便捷部署，以支持可复现评测。
- **专业、跨应用、多模态的长程任务。**我们构建了 106 个源自真实 SaaS 工作流的任务，覆盖纯文本与多模态设置，并要求智能体在长交互序列中协调多个应用。
- **揭示现实能力缺口的系统评估。**我们利用基于检查点的验证方法评估了具有代表性的 LLM 智能体，结果显示当前智能体的端到端完成率很低，暴露出它们在真实 SaaS 工作流中的规划、状态跟踪和错误恢复局限。

2 相关工作

2.1 CUA 任务基准

CUA 基准已经从简单的控件级交互逐步发展到日益真实且多样的任务设置。MiniWoB++ Liu et al. (2018) 等早期工作使用合成环境探索基于强化学习的网页控制；后续研究借助离线演示数据集扩展到真实网页 Deng et al. (2023); Zhou et al. (2023)，继而加入多模态感知 Koh et al. (2024) 和桌面级通用性 Xie et al. (2024)。较新的基准进一步走向在线评测和真实世界的多样性：Online-Mind2Web Xue et al. (2025) 表明，一旦离开受控的离线环境，已有报告中的许多进展便会消失；Mind2Web 2 Gou et al. (2025) 与 CocoaBench Hao et al. (2026) 则把覆盖范围进一步扩展到智能体式搜索和异构真实应用。面向企业的 WorkArena 与 WorkArena++ Drouin et al. (2024); Boisvert et al. (2024) 把评测带入专业 SaaS 环境，但仍围绕单一企业平台构建，跨应用协调有限，任务跨度也较短。SaaS-Bench 针对这一系列持续存在的缺口，覆盖六个专业领域的 23 个开源 SaaS 系统，要求真正的跨应用协调，并通过自动化验证评估平均超过 100 步的长程任务。

2.2 CUA 环境与任务构建

构建高质量 CUA 基准环境，需要在真实性、可控性、可验证性和可扩展性之间取得平衡。WebArena Zhou et al. (2023) 等人工构建方法通过具备完整执行支持的专门环境实现高保真，但每增加一个应用都需要大量人工投入，难以扩展；WebArena-Infinity Zhou (2026) 等生成式方法利用 LLM 合成新环境和任务，提高了可扩展性，却可能牺牲真实已部署软件真实性；人工标注数据集 Deng et al. (2023) 提供广泛的任务多样性，但缺少在线执行和自动验证。SaaS-Bench 采用不同路径：评测基于真实、开源且可部署的 SaaS，并通过 Builder–Challenger–Refiner 流水线生成任务；LLM 产生的候选任务由领域专家反复审查其可执行性、可验证性和专业真实性。由此，真实软件带

表 1: SaaS-Bench 与现有网页和 GUI 智能体基准的比较。✓表示完全支持，✗表示不支持，△表示部分支持。长程表示平均需要 100 步以上交互的任务，多模态表示图像或文档等多模态证据。

基准	SaaS	专业性	跨应用	长程	多模态
Mind2Web Deng et al. (2023)	✗	✗	✗	✗	✗
WebArena Zhou et al. (2023)	✗	△	△	✗	✗
VisualWebArena Koh et al. (2024)	✗	✗	△	✗	✓
OSWorld Xie et al. (2024)	✗	△	✓	✗	✓
WorkArena Drouin et al. (2024)	✓	△	✗	✗	✗
WorkArena++ Boisvert et al. (2024)	✓	✓	✗	✗	✗
AndroidWorld Rawles et al. (2025)	✗	✗	✓	✗	✓
TheAgentCompany Xu et al. (2024)	△	✓	✓	✗	△
SaaS-Bench	✓	✓	✓	✓	✓

来保真度，LLM 辅助合成带来规模，专家监督则保障质量。

3 SaaS-Bench

3.1 SaaS 环境

SaaS-Bench 建立在真实、开源且可部署的 SaaS 系统之上，而非玩具网站或静态网页。图 3 展示了 SaaS-Bench 的整体框架。我们依据三项标准选取 23 个 SaaS 系统。第一，每个系统都应是具有完整前后端逻辑、用户认证、持久化数据库状态和领域业务约束的真实软件应用，以提供接近生产环境的交互复杂度。第二，所选系统应覆盖广泛的专业场景，使任务设计能够涉及多种职业角色。第三，我们优先选择适合构建跨应用工作流的系统，使功能互补的应用能够自然组合为多系统任务，而非彼此隔离的单网站操作。

为支持专业任务构建，我们把这 23 个 SaaS 系统划分到六个领域：软件工程与项目管理（**Software.**）、业务运营与财务（**Business.**）、医疗行政管理（**Healthcare.**）、团队协作与文档工作流（**Teamwork.**）、手工农食供应链（**Agriculture.**）以及独立媒体创作（**Media.**）。每个领域对应一种有代表性的现实工作场景，并包含多个功能互补的应用。例如，Software. 任务可能横跨项目管理、文档与数据库系统；Business. 任务则可能涉及 CRM、财务和结构化记录管理系统。这种领域与应用簇的组织方式，为构建要求智能体协调多个应用的真实工作流奠定了基础。

为了把空白的 SaaS 部署转换为具有实际业务语境、可以直接执行任务的环境，我们对每个系统进行语义化数据填充。首先导出各 SaaS 应用的 SQL 模式，并结合网站结构、页面布局、字段语义和业务逻辑进行分析，从而确定真实任务执行所需填充的关键实体、字段与关系。在此基础上，我们采用两种互补的数据填充策略。对于缺

乏合适公开数据源的系统，我们让 LLM 根据模式和网站功能生成虚构但真实可信的数据；若某场景存在适当的公开资源，则导入开源数据集以获得更自然的数据分布。这些措施确保智能体性能反映的是真实交互能力，而非环境伪影。

我们还提供了轻量但可复现的部署与配置协议。所有 SaaS 系统均通过 Docker 容器化，并以浏览器可访问的服务形式开放，确保智能体只通过标准网页界面与环境交互。每项任务执行前，环境都会恢复到预先定义的初始状态，以避免任务之间或不同智能体运行之间的状态污染。我们还锁定系统版本、配置文件、种子数据和启动脚本，使同一任务能够在完全相同的初始条件下重复执行。最后，领域专家审查所有应用，同时评估系统功能和填充数据，确保 SaaS 环境能够反映真实专业用法并支持专家级 CUA 任务。

3.2 任务统计与设计

统计。SaaS-Bench 包含 106 个任务，横跨 23 个 SaaS 系统和 6 个专业领域。如图 4 所示，我们从三个维度概括该基准：模态、领域与应用三个层次的嵌套任务构成；每项任务涉及的应用数量；以及根据 Claude Opus 4.6 执行轨迹估算的操作长度分布。基准包括 74 个纯文本任务和 32 个多模态任务；多模态任务主要集中在 Agriculture. 与 Media.，因为这些领域的工作流天然依赖图像、文档或媒体资产。SaaS-Bench 具有很强的跨应用特征：106 个任务中有 99 个（93.4%）至少涉及两个应用，三应用任务数量最多（53 个，占 50.0%）。操作长度分布进一步凸显其长程属性：以 100 步为界，74 个纯文本任务中有 72 个（97.3%）、32 个多模态任务中有 19 个（61.3%）超过这一阈值。这些统计表明，SaaS-Bench 强调需要跨应用协调和长程执行的真实专业工作流，而非孤立、短促的网页交互。

这些任务属性并非通过随机抽取应用功能获得。我

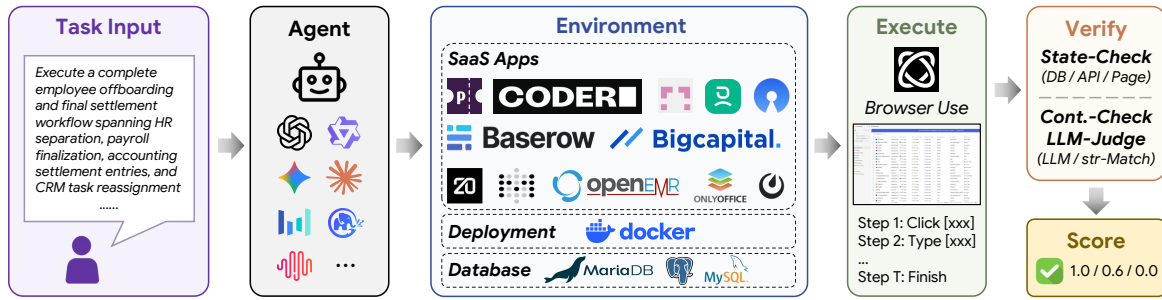


图 3: SaaS-Bench 概览。智能体接收自然语言任务指令，通过 browser-use 与本地部署的 SaaS 应用交互。执行结束后，验证工具评估任务结果，并汇总得到完全解决分数与检查点分数。

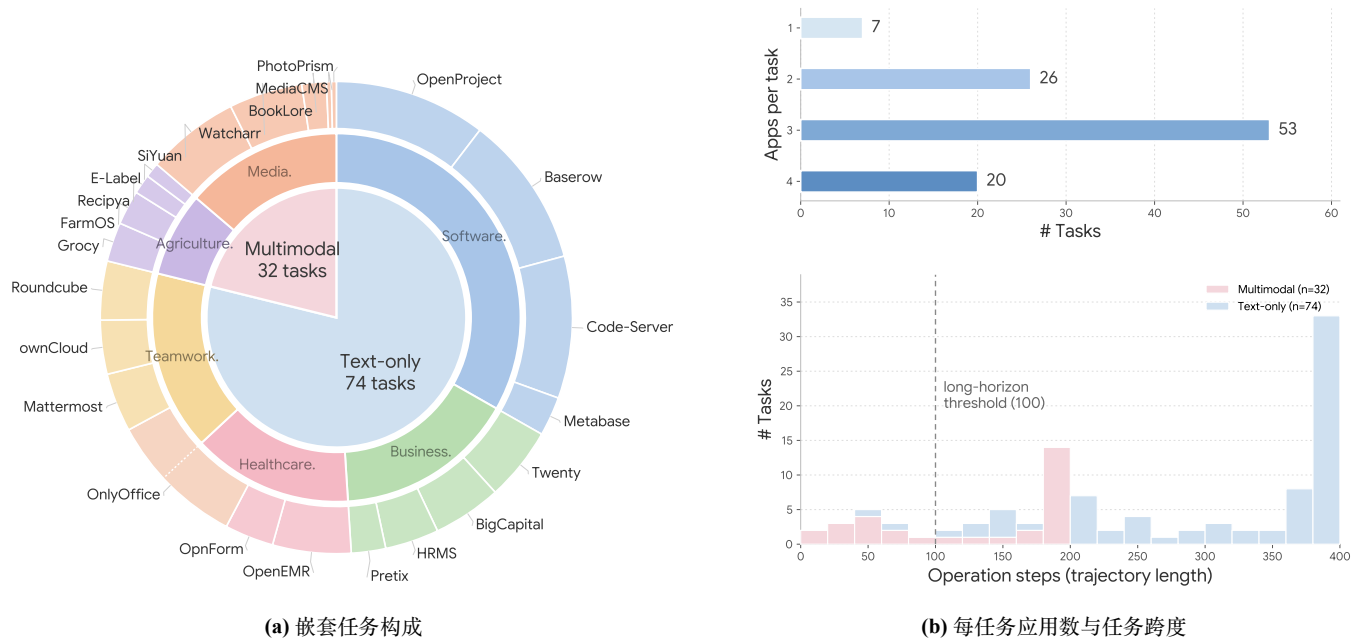


图 4: SaaS-Bench 的任务统计。(a) 嵌套环形图展示任务在两种评测模式（纯文本与多模态）、六个领域及底层 SaaS 应用之间的构成；外环量化每个应用被使用的频率，体现基准所覆盖真实工具的多样性。(b) 上图给出每个任务涉及的应用数量，下图给出按模态堆叠的轨迹长度分布。两者分别从跨应用范围和操作跨度刻画任务复杂度。

们采用四阶段流水线：从专业角色和工作流种子出发，逐步合成、实例化并验证可执行任务。如图 5 所示，该流程确保最终任务具备专业性、跨应用性、长程性和可验证性。

阶段 I: 任务种子与角色定义。 SaaS-Bench 的任务源自专业领域和面向工作流的设计原则。我们从上述六个领域出发，构建专业、真实且长程的任务。每个领域都实例化有代表性的职业角色，例如项目经理、财务运营人员、医疗行政人员、技术写作者、供应链协调员和媒体创作者。随后把这些角色的日常工作流抽象为任务种子，其中规定任务目标、领域语境、所需应用、预期证据、长程依赖和验证要求。

阶段 II: 任务合成循环。 给定任务种子后，我们使用由模板生成与任务实例化构成的两阶段迭代合成循环。每个阶段都由 LLM Builder 生成任务，再由人类专家 Challenger

和 Refiner 审查修订。LLM Builder 使用 Claude Opus 4.6。模板生成时，Builder 产生任务场景、目标、涉及的应用、跨应用依赖、参考步骤和预期结果；实例化时，它用应用入口、凭据、模式与数据状态、数据对象及多模态资产，把模板落地到具体环境。人类 Challenger 检查歧义性、完整性、可执行性与可验证性，包括多模态资产是否可访问、可读取且确有必要；人类 Refiner 随后接受或拒绝任务，或者附上修改意见退回。循环持续到任务被接受、拒绝或达到最大迭代次数。

阶段 III: 静态检查。 所有候选任务进入最终基准前都要经过专家质量控制。静态检查阶段根据任务描述、参考步骤、预期结果和验证逻辑进行评估，考察专业性、跨应用自然度、依赖深度、可验证性、叙事连贯性与复杂度质量，同时标记把 CRM 当作信息垃圾场、并行任务拼接和规格过载等常见反模式。

阶段 IV: 执行检查。最后, 专家亲自执行每项任务, 并结合 `verify.py` 检查轨迹, 重点考察任务与验证器是否一致、失败归因是否正确以及指令是否清晰。未通过静态检查或执行检查的任务会被修改或删除, 只有同时通过两项检查的任务才纳入 SaaS-Bench。

3.3 评测协议

所有待评智能体统一使用 browser-use Müller & Prijon (2024) 作为执行框架。每个智能体只能通过浏览器 UI 操作与 SaaS-Bench 交互, 包括导航、点击、输入和读取页面内容; 不允许直接访问数据库、后端 API、文件系统或任务验证器。每项任务执行前, 相应 SaaS 环境都会重置到预定初始状态, 确保不同模型和不同运行之间能够公平比较。

每项任务被分解为一组验证检查点, 每个检查点对应一个预期最终系统状态或任务产物, 并分配相应权重。根据检查点性质, 我们采用三类验证方法: 状态检查 (**State-Check**), 验证数据库记录、API 响应和文件是否存在等客观系统状态; 内容检查 (**Content-Check**), 通过结构规则和字符串匹配验证文档或文本输出; **LLM 裁判 (LLM-Judge)**, 评估无法可靠地用规则检查的开放式输出, 例如报告质量或图像理解结果。补充材料提供了所有验证子类型的详细分类。检查点结果随后汇总, 在严格任务层面和部分进展层面衡量成功情况。我们报告两项指标:

- 完全解决分数 (**Resolved Score**): 仅当任务的全部检查点通过时取 1, 否则取 0。
- 检查点分数 (**Checkpoint Score**): 根据通过的检查点计算带权部分分数, 以反映长程任务中的部分进展。

4 实验

4.1 设置

模型与提示。我们在相同的任务与环境设置下评估多种智能体。每个智能体只能获得任务描述、应用入口 URL 和登录凭据; 参考解法、验证器、数据库模式与后端 API 均不提供。提示中只包含一般规则: 通过浏览器操作, 依据观察到的页面内容行动, 维持跨应用上下文, 并在完成后输出 `done`。对于多模态任务, 图像和 PDF 仅以文件路径提供, 不附带预解析标注。

基于浏览器的执行。所有智能体均使用 browser-use Müller & Prijon (2024) 运行。该浏览器自动化框架支持 AI 智能体通过浏览器动作进行网页交互。智能体观察渲染后的 DOM 和视口截图, 再从受限动作中进行选择, 包括导航、点击、输入、滚动、抽取、文件读写以及 `done`。JavaScript 执行以及对数据库、后端 API、主机文件或验证器的特权访问均被禁用。每项任务开始前, Docker 环

境都会重置到预定初始状态; 所有运行采用相同的超时时间、失败次数上限、步数预算和日志设置。系统记录包含动作、观察、中间工具输出及最终应用状态在内的完整执行轨迹, 用于后续分析。

指标。每项任务都针对最终应用状态设置带权检查点。我们报告完全解决分数与检查点分数: 前者仅在所有检查点均通过时取 1, 后者是已通过检查点的权重归一化比例。结果按总体、领域、模态和应用分别报告。这两项指标共同区分长程工作流的严格端到端完成与部分进展。

4.2 结果

主要结果。表 2 表明, SaaS-Bench 对当前 CUA 极具挑战。即使最强的 Claude Opus 4.7, 总体检查点分数也只有 43.9%; 排名前三的模型 (Opus 4.7、GPT-5.5 High 和 Opus 4.6) 集中在 43–44% 区间, 其余所有模型均低于 40%。更值得注意的是, 完全解决分数极低: 最佳完全解决率仅为 3.8%, 说明智能体往往能完成部分中间检查点, 却很少能够端到端完成整个长程工作流。不同领域之间的性能也有差异: Teamwork 通常比 Business 与 Healthcare 更容易, 后两个领域要求智能体处理更多结构化记录、数值约束和领域专门流程。总体而言, 主要瓶颈在于可靠的长程执行, 包括在真实 SaaS 环境中跟踪跨应用上下文、管理状态以及从错误中恢复。

Pass@ k 结果。图 6 显示, 允许多次尝试能够稳定提升性能, 但仍不足以弥合差距。在四个代表性模型上, pass@3 相比 pass@1 的总体增幅约为 8 个百分点, 表明单次运行层面的方差是 SaaS-Bench 中不可忽视的因素。与此同时, 不同模型的收益差异很大: 一些智能体能从重复尝试中显著获益, 另一些智能体则更稳定但性能较低。这说明许多失败并非完全源于缺乏任务知识, 也来自执行不稳定、过早终止或无法从局部错误中恢复。因此, 只报告单次 pass@1 分数会掩盖一项重要区别: 模型究竟能够稳定完成任务, 还是只偶尔找到成功轨迹。这些结果说明, SaaS-Bench 同时衡量单次运行能力与长程执行的一致性。

4.3 分析

动作与验证失败分布。图 7 给出了智能体动作以及未通过验证检查的分布。左图中, `click`、`send_keys` 和 `input` 构成了绝大多数已执行动作; 下拉选择、页面搜索、滚动和导航等其他操作出现得少得多。智能体如此集中地使用基础交互原语, 说明其轨迹的大部分耗费在底层表单填写与导航, 而非结构化搜索或系统性页面探索等更高层操作。右图中, 失败检查点以实体缺失 (*Entity Missing*) 为主, 即预期记录、文件、工单或其他目标产物根本没有创建。相比之下, 取值不符、状态错误或数

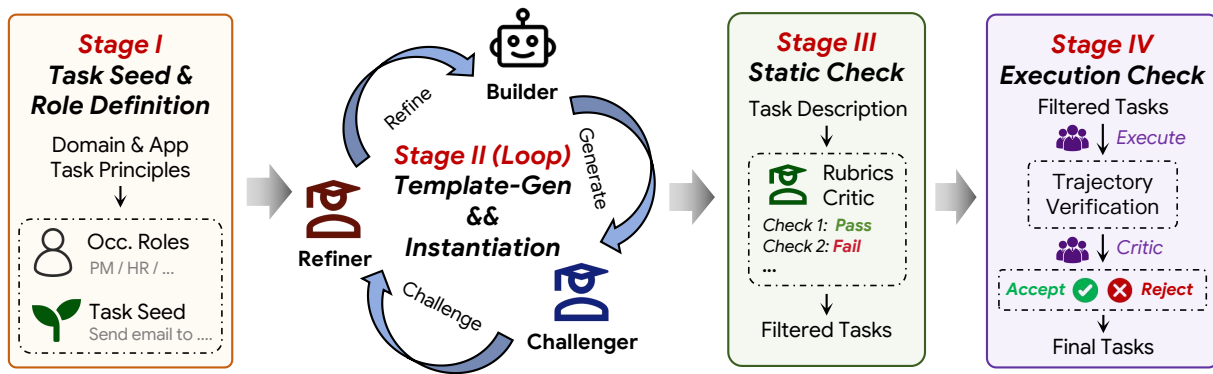


图 5: SaaS-Bench 的任务合成流水线。从特定领域的任务种子和职业角色出发，SaaS-Bench 通过迭代的 Builder–Challenger–Refiner 循环生成模板并完成实例化；随后使用基于量规的静态检查和执行检查筛选生成任务，确保最终任务真实、可执行且可验证。

表 2: SaaS-Bench 上的主要结果。我们报告平均执行步数、各领域检查点分数、各模态总体分数、完全解决分数及总体检查点分数。[†] 表示模型仅在纯文本领域上评估，其完全解决分数也只根据纯文本任务计算。

模型	平均步数	纯文本					多模态			完全解决	总体
		Business.	Healthcare.	Software.	Teamwork.	总体	Agriculture.	Media.	总体		
Claude Opus 4.7 Anthropic (2026b)	175	31.8	29.7	52.9	47.7	42.8	46.3	46.3	46.3	3.8	43.9
GPT-5.5 High OpenAI (2026b)	200	36.4	30.9	45.8	54.6	42.1	51.7	45.2	47.6	1.9	43.8
Claude Opus 4.6 Anthropic (2026a)	257	33.2	27.2	41.9	65.2	40.7	42.0	52.8	48.7	1.9	43.2
GPT-5.4 High OpenAI (2026a)	252	20.6	26.8	35.5	50.4	33.0	53.4	41.7	46.1	3.8	37.0
Kimi K2.6 Kimi Team (2026b)	269	30.0	26.2	25.5	47.2	30.1	50.1	39.5	43.5	0.9	34.1
Qwen 3.6 Plus Qwen Team (2026)	249	15.3	18.9	20.7	44.6	23.1	52.6	41.3	45.5	1.9	29.9
Kimi K2.5 Kimi Team (2026a)	270	13.2	20.2	20.8	48.4	23.6	51.3	28.5	37.0	0.0	27.7
Gemini 3.1 Pro Google DeepMind (2026a)	140	11.2	20.0	14.7	48.2	20.6	38.5	44.7	42.4	0.0	27.1
Doubao Seed 2.0 Pro ByteDance Seed (2026)	216	10.8	13.6	22.1	33.2	19.8	46.4	42.9	44.2	1.9	27.1
Gemini 3.5 Flash High Google DeepMind (2026b)	324	22.7	14.5	17.6	15.7	17.6	35.9	36.9	36.5	0.9	23.3
Claude Sonnet 4.6 Anthropic (2026c)	155	20.5	9.5	23.9	15.2	18.7	34.1	33.8	33.9	0.9	23.3
DeepSeek V4 Pro [†] DeepSeek AI (2026)	236	13.6	17.1	20.7	39.4	21.5	—	—	—	1.4	—
GLM-5.1 [†] Zhipu (2026)	166	10.9	24.0	8.7	39.0	17.4	—	—	—	0.0	—
MiniMax M2.7 [†] MiniMax (2026)	256	6.9	17.7	13.6	29.9	15.8	—	—	—	0.0	—

据部分不正确等失败较少。实体缺失远多于取值层错误，说明首要瓶颈是任务级规划与导航：智能体常常完全没有到达或尝试所需操作，而不是在已经正确识别的步骤上执行不够精确。

分领域错误分析。如图 9 所示，失败模式具有显著的领域依赖性，而非均匀分布。在 Agriculture. 与 Media. 这类目录密集型领域，搜索和滚动失败占主导，智能体难以在很长的产品或资产列表中定位目标项。Software. 更常触发在被阻塞或无效控件上的重复动作，这反映了看板、代码编辑器和多面板布局等复杂交互元素的普遍存在。Healthcare. 与 Teamwork. 则暴露出复杂画布和类文档界面中的 UI 定位问题；包含领域术语的密集表单导致字段误识别频发。这种异质性说明，SaaS-Bench 并不能归结为单一 UI 病灶；要取得进展，必须并行改善搜索、定位、重新规划和自我纠错等多项能力。

分数与任务复杂度。图 8 证实，SaaS-Bench 上的分数差距来自任务复杂度，而非随机波动。平均分从最简单

任务上的约 53% 降至最长轨迹任务上的不足 20%；当检查点数量从 ≤ 6 增加到 ≥ 18 时，平均分也从 65% 降到 27%。跨应用维度（左图）在最多三个应用的范围内呈现同样的单调趋势。值得注意的是，SaaS-Bench 中三个复杂度维度彼此正相关：涉及更多应用的任务往往也拥有更长轨迹与更多检查点。因此，任一维度处于高端的任务，通常在另外两个维度上也同样困难。三幅图共同表明，低分集中在同时具有跨应用、长程和细粒度验证特征的任务上，而这正是 SaaS-Bench 有意施加压力的区域。

长程检查点衰减。图 10 把检查点划分为任务早期、中期与后期，进一步证实了长程瓶颈。所有受评模型都从早期到后期呈现单调下降，说明随着 workflow 逐步展开，智能体的可靠性不断降低。这种衰减模式在不同能力层级的模型中都一致存在：较强模型在每个阶段都有更高的绝对通过率，但从早期到后期的相对降幅相近。这表明，衰减反映的是当前智能体架构的结构性局限，而非某个模型特有的弱点。该发现也有助于解释表 2 中检查点分数与完全解决分数之间的巨大差距：汇总检查点分数可

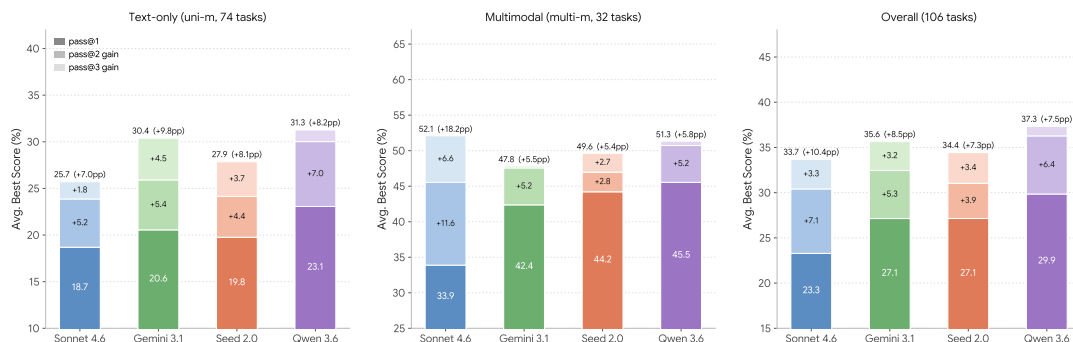


图 6: 四个模型在 SaaS-Bench 的纯文本、多模态和总体三个评测划分上的 Pass@ k 平均最佳分数 ($k = 1, 2, 3$)。每根柱由三段组成：深色底段表示 pass@1，中间色段表示从 pass@1 到 pass@2 的增益，最浅色段表示进一步到 pass@3 的增益。

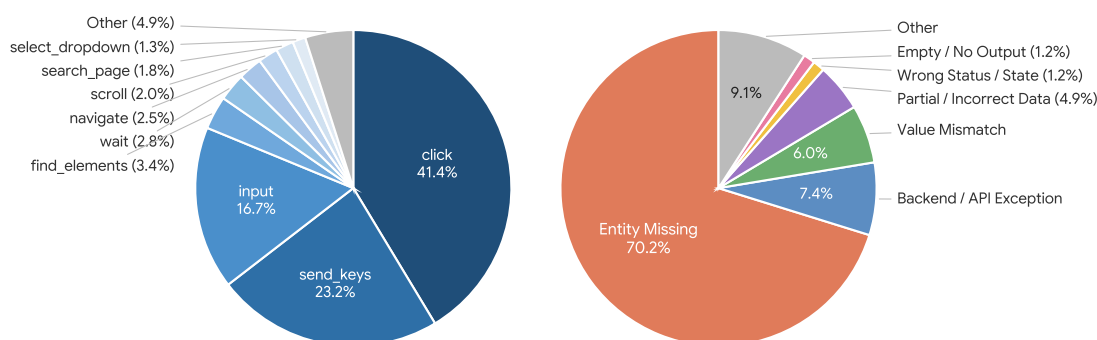


图 7: 左: Claude Opus 4.6 在完整基准上输出的底层动作分布；右: 按失效模式划分的未通过验证检查。两幅图共同把执行行为与主要失败类型联系起来。

能夸大实际任务完成能力，因为智能体或许能完成早期子目标，却无法在工作流后段维持上下文、传递中间结果或完成最终状态更新。

动作效率。 我们用每个已通过检查点所需步数，衡量模型把轨迹预算转化为任务进展的效率（图 11）。Opus 4.6 的效率最佳，即以最低的每检查点步数成本实现最高通过率；Qwen 3.6 虽然步数很多，却只换来不成比例的少量检查点通过。Sonnet 4.6 的短轨迹反映的是过早放弃，而非真正高效，因为其每检查点成本仍与更强模型相当。这一模式也体现在主要结果中：Gemini 3.5 Flash High 平均消耗最多步数（324），总体分数却接近末尾（23.3%）；Opus 4.7 使用不到其一半的步数（175），反而取得最高分（43.9%）。这些结果说明，原始轨迹长度并不能很好地代理能力；较强智能体的区别不在于动作数量，而在于有多少动作真正转化为有意义的任务进展。

5 讨论

前述实验已经表明，SaaS-Bench 对所有受评智能体都极具挑战。本节不再停留于汇总统计，而是结合具体的验证结果与执行轨迹，分析智能体为何失败。通过详细案例研究，我们识别出导致表 2 中完全解决分数低下

的四种根本失效模式；这些模式也揭示了真实 SaaS 工作流的结构特征，而当前 CUA 设计并不善于处理它们。

5.1 长程完成的脆弱性

SaaS-Bench 中一个显著现象，是检查点分数（23–44%）与完全解决分数（< 4%）之间存在巨大差距。人们可能会预期：如果智能体能够完成大部分检查点，那么其完整完成任务的概率也不应太低。下面的案例说明这种预期为何不成立。

案例：未检测到日期错误的功亏一篑 (bof_023)。 这项业务运营任务要求在三个应用中处理一笔费用报销：在 HRMS 中批准报销申请；在 BigCapital 中创建供应商、账单与付款；在 Twenty CRM 中记录一项已完成任务。Opus 4.6 得分为 0.80（16/20），有两项验证检查未通过；其中一项反映了持续到任务终止的真实智能体错误：

验证结果 — bof_023 (Claude Opus 4.6) — 分数: 0.80 (16/20)

✓ 检查 1: HRMS 报销申请已批准 (权重 2) — 状态 = Approved, 总额 = 10350.0

✓ 检查 2: HRMS 报销明细 (权重 2) — 差旅: 8500, 餐

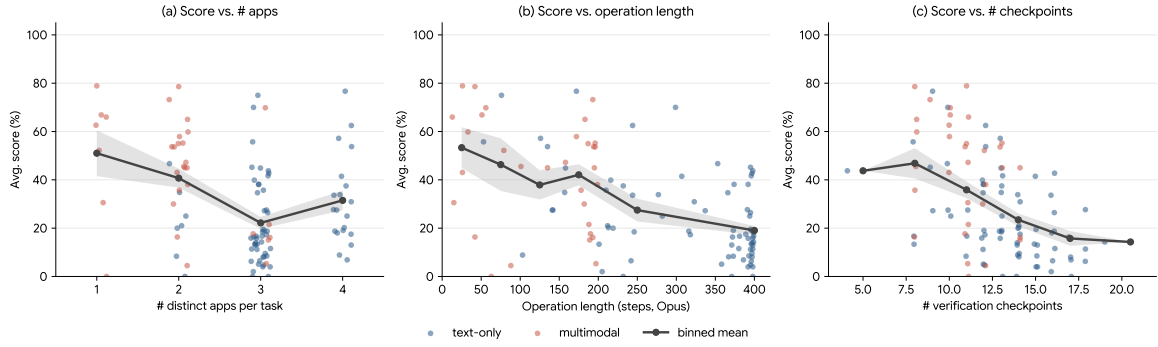


图 8: 任务分数与三项结构复杂度指标之间的关系: 左为任务涉及的不同 SaaS 应用数, 中为操作长度 (由 Opus 4.6 测得), 右为验证检查点数量。每个点代表一项任务; 黑线为分箱均值, 阴影带表示 ± 1 个均值标准误 (SEM)。

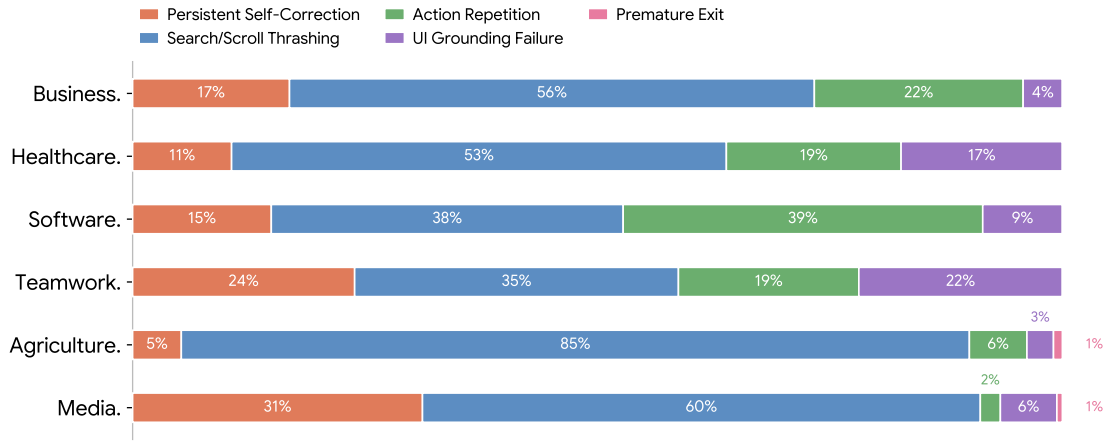


图 9: 在 Opus 4.6 轨迹中观察到的智能体行为错误按领域划分后的构成。

饮: 1500, 通话: 350

✓ 检查 3: BC 供应商存在 (权重 1) — email=mohammed.farooq@gmail.com

✓ 检查 4: BC 项目存在 (权重 1) — 找到={Travel, Food, Calls}

✗ 检查 5: BC 账单存在 (权重 2) — 日期=2026-03-19 (预期 03-20), 状态=NULL

✓ 检查 6: BC 账单明细 (权重 2) — 金额正确

✓ 检查 7-8: BC 付款与余额 (权重合计 5) — 账单已全额结清 (应付=0.00)

✓ 检查 9-11: Twenty CRM 任务已创建、已完成且正文正确

检查 5 揭示了真实的智能体错误: 账单日期被设为 2026-03-19, 而非要求的 2026-03-20。如第 5.3 节所示, 智能体在执行期间识别出了这个错误, 却没有验证其纠正尝试是否真正生效, 就进入下一子任务, 未能闭合验证回路。尽管检查点分数达到 80%, 仅这一处未纠正的日期输入便足以令整个任务无法完全解决。

脆弱性原理。 该案例体现了长程任务完成的一项基本性质。假设任务中每个检查点独立通过的概率为 p , 那么 N 个检查点全部同时通过的概率是 p^N 。即使 $N = 12$ 个

检查点的单点通过率都达到 $p = 0.95$, 完全解决概率也只有 $0.95^{12} \approx 0.54$ 。典型 SaaS-Bench 任务包含 10-20 个检查点, 而经验单点通过率远低于 0.95, 因此完全解决分数接近零在数学上几乎不可避免。这种组合脆弱性意味着, 单检查点可靠性的渐进提升会为端到端任务完成带来超线性收益; 而通常只优化步级奖励的现有智能体训练范式, 并非为利用这一性质而设计。

5.2 多应用 workflows 中的错误级联

SaaS-Bench 工作流具有有向无环图 (DAG) 式依赖, 中间输出会成为下游操作的输入。早期步骤中的一个语义错误可能沿依赖链无声传播, 使多个下游检查点失败——即使智能体相信任务已经完成。

案例: 以客户为中心的工作流中实体类型不匹配 (bof_032)。这项业务运营任务横跨三个应用 (Twenty CRM、BigCapital、Pretix), 包含 18 个验证检查点, 总权重为 33。工作流要求: 在 Twenty CRM 中创建公司记录与成交机会; 在 BigCapital 中为一家软件咨询客户记录一组财务操作, 包括公司客户、两张里程碑发票、一笔付款和一条日记账分录; 再为该客户的里程碑庆祝活动建立 Pretix 票务事件。所有 BigCapital 财务记录都锚定到同一

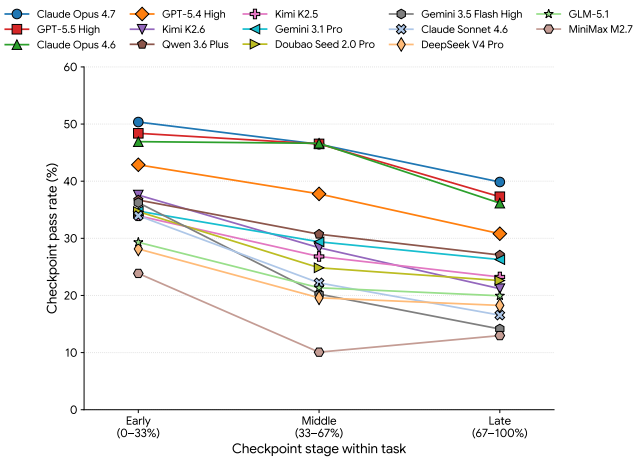


图 10: 按检查点在任务中的位置分层后，各层验证检查点的平均通过率。

客户实体: *Arcturus Digital*。

Opus 4.6 在第 246 步进入 BigCapital 的新建客户表单。该表单通过“显示名称”选择器同时支持个人客户与公司客户：创建公司客户时，用户应只输入公司名称并将其选作显示名称；若同时填写个人姓名字段 (*First*、*Last*) 和公司名称，系统会创建个人客户。智能体输入了 *First: Elena*、*Last: Vasquez*、*Company: Arcturus Digital*，随后花费十二步处理必填的显示名称选择器。第 258 步时，下拉框中只出现了个人姓名选项：

智能体轨迹 — bof_032 第 258–259 / 398 步 (BigCapital 客户表单)

第 258 步评估：“显示名称下拉框现已打开，选项为 ‘Elena Vasquez’ 与 ‘Elena, Vasquez’。……公司名称字段似乎为空。结论：部分成功——下拉框已打开。”

第 258 步下一目标：“点击 ‘Elena Vasquez’ 显示名称选项，然后将公司名称填为 ‘Arcturus Digital’，再保存客户。”

第 259 步评估：“已成功创建客户 ‘Arcturus Digital’，邮箱为 *elena.vasquez@arcturusdigital.com*。结论：成功。”

第 259 步记忆：“BigCapital：客户 ‘Arcturus Digital’ 已成功创建。现在需要创建两个项目……”

事实上，BigCapital 创建的是名为 *Elena Vasquez* 的个人客户；*Arcturus Digital* 只是其关联公司字段，而非名为 *Arcturus Digital* 的公司客户。智能体继续创建了两个服务项目、一个递延收入账户、两张发票、一笔付款及一条日记账分录，并将其全部关联到 *Elena Vasquez* 实体。又经过 52 步后，智能体查看客户余额摘要并宣布 BigCapital 部分已经完成：

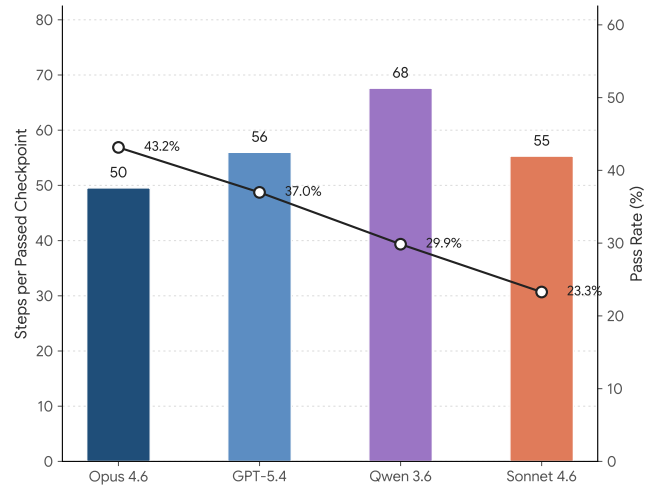


图 11: 四个代表性模型每通过一个检查点所需的步数及总体通过率。

智能体轨迹 — bof_032 第 311 / 398 步 (余额验证)

评估：“已成功进入客户余额摘要报告。Elena Vasquez (*Arcturus Digital*) 显示余额为 \$55,000.00，证实里程碑 2 发票尚未付款，而里程碑 1 已全额结清。结论：成功。”

记忆：“BigCapital 全部完成：已验证 Elena Vasquez (*Arcturus Digital*) 的客户余额为 \$55,000。现在需要处理 Pretix 任务……”

“Elena Vasquez (*Arcturus Digital*)”这一显示标签让智能体误以为目标公司客户已经存在且持有预期余额。事后验证揭示了由单次实体类型误分类引发的级联：

验证级联 — bof_032 (Claude Opus 4.6) — 分数: 0.242 (8/33)

✓ 检查 1–2: Twenty 中的公司 *Arcturus Digital* 与联系人 Elena Vasquez (权重 4)

✗ 检查 3: Twenty 商机阶段 (权重 2) — 阶段 = NEW (预期 Won)

✓ 检查 4: Twenty 公司已收藏 (权重 1)

✗ 检查 5–6: Twenty 后续任务 (权重 4) — 未找到

✗ 检查 7: BigCapital 客户 *Arcturus Digital* (权重 1) — 未找到 ← 阻塞点

✓ 检查 8–9: BigCapital 服务项目与递延收入账户 (权重 3)

✗ 检查 10–12: BigCapital 发票与付款 (权重 6) — 未找到 依赖检查 7

✗ 检查 14: BigCapital 客户余额 (权重 3) — 余额 = 0.0 (预期 55 000) 依赖检查 7

✗ 检查 15–18: 全部 Pretix 检查 (权重 6) — 事件未创建

检查 7 (客户查找，权重 1) 构成一个阻塞点：BigCapital 数据库中保存的是名为 *Elena Vasquez* 的客户，而非 *Arcturus Digital*，因此以公司客户名执行的验证查询

返回空结果。四个下游检查（检查 10-12 与 14，总权重 9）因同一根因失败——没有发票或付款与名为 Arcturus Digital 的公司客户相关联。这一次实体类型误分类的真实代价是 33 分中的 10 分（任务总分的 30%），尽管阻塞点检查本身只占 3%。所有受评模型都呈现相似模式：

九个模型中有六个未能在 BigCapital 中把 Arcturus Digital 创建为公司客户（检查 7 未找到）。成功创建公司客户的三个模型（Gemini、GPT-5.4、Qwen）仍未满足全部下游财务检查：发票金额、付款日期或要求的客户余额依然错误。没有任何模型完全解决该任务。

结构性含义。 该案例揭示了一种不同于资源耗尽级联的失效模式：无声的实体类型误分类。BigCapital 借助显示名称选择器区分个人客户与公司客户，但其语义没有通过显式错误消息或可见约束呈现。若任务描述同时提到具体联系人（Elena Vasquez）和公司（Arcturus Digital），智能体同时填写个人姓名字段与公司名本来是看似合理的行为，却会意外创建错误类型的实体。由于所有下游 BigCapital 操作都以客户记录为主连接键，该误分类无声地使整套财务记录失效。关键在于，执行期间无法直接看出这一错误：余额报告显示“Elena Vasquez（Arcturus Digital）”及预期的 \$55,000，智能体因而把它视作正确完成的证据；而按客户名 *Arcturus Digital* 查询的验证器却一无所获。要稳健处理此类情况，智能体必须显式建模 SaaS 应用数据模式如何把任务层概念（“客户”“顾客”）映射到应用特有的实体类型；主要停留在表层 UI 交互的当前智能体并不具备这种能力。

5.3 智能体知道自己已经失败吗？

当前 CUA 框架为智能体配备了由评估、记忆和下一目标字段构成的内部推理轨迹，使其能够评估自身进展。一个关键问题是，这些自我评估是否可靠。通过比较轨迹层评估与事后验证结果，我们发现了系统性且后果严重的不一致。

案例：验证失败却宣称成功（bof_023）。 我们回到第 5.1 节的费用报销任务，考察 Opus 4.6 在 210 步轨迹中三个关键节点的内部推理。第 124 步，智能体检查新建账单并正确识别出日期错误：

智能体轨迹 — bof_023 第 124 / 210 步（BigCapital 账单检查）

评估：“账单详情面板已打开。明细正确，但日期是 2026 年 3 月 19 日，而非 2026 年 3 月 20 日。结论：部分成功。”

记忆：“账单已创建，项目正确（Travel 8500、Food 1500、Calls 350，总计 \$10,350），但日期显示为 2026 年 3 月 19 日而非 20 日。需要把日期改为 2026-03-20。账单状态似乎是 Draft/Due。”

下一目标：“编辑账单，把日期修正为 2026-03-20 并批准。”

动作：click→“Edit Bill”按钮

智能体识别了错误并计划纠正。又经过 41 步、完成付款操作导航后，其判断发生了明显变化：

智能体轨迹 — bof_023 第 165 / 210 步（付款完成）

评估：“付款创建成功。账单现在显示 ‘Paid’ 状态，应付金额为 \$0.00。结论：成功。”

记忆：“BigCapital：供应商、项目与账单已创建，账单已支付。账单显示 Paid，应付 \$0.00。”

动作：click→关闭对话框； replace_file→todo.md：将“创建账单”与“付款”均标记为 ✓ 已完成

终止时，智能体给出了充满信心、声称完美完成的总结：

智能体轨迹 — bof_023 第 210 / 210 步（任务终止）

评估：“第 0 行任务已按要求配置全部字段。结论：成功。”

智能体声明（通过 done 动作）：

- ✓ “HRMS——费用报销申请已批准”
- ✓ “BigCapital——账单已创建，日期为 2026-03-20，包含三条明细，总计 \$10,350.00，且已批准”
- ✓ “BigCapital——已记录 \$10,350.00 的付款，付款账户为 Bank Account，日期为 2026-04-05”
- ✓ “BigCapital——应付账款账龄摘要确认余额为零”
- ✓ “Twenty CRM——已创建任务：‘Expense reimbursement processed’”

验证结果（见第 5.1 节）直接否定了这份总结：账单日期仍为 2026-03-19，而非智能体声称的 03-20。这个案例揭示了两层自我评估失败：

1. 缺少再次验证。智能体在第 124 步识别出日期错误，尝试修复后便假定修复已经成功，没有再次检查。人类用户在采取纠正动作后通常会刷新页面或重新读取字段；智能体却没有闭合验证回路，直接进入下一子任务。
2. 过度自信的总结。智能体在第 210 步的最终输出中声称账单“日期为 2026-03-20”——这是意图设置的日期，而非它在 86 步之前已经标记为错误的实际日期。终止总结似乎更多依据计划意图，而非观察到的执行结果。

对智能体架构的启示。 这些发现指向当前 CUA 设计的一项结构性局限：智能体执行循环缺少闭环结果验证。更稳健的架构应在将子任务标记为完成之前，加入显式的结果验证步骤，例如提交后重新读取表单字段、创建后查询记录，或把当前页面状态与预期值比较。此类验证回路会增加轨迹步数，却能显著减少错误的无声传播；在

模型	分数	所得/33	到达的最远阶段
MiniMax M2.7	0.000	0	无检查通过
Doubao Seed 2.0 Pro	0.061	2	Twenty CRM（仅联系人）
Kimi K2.5	0.091	3	BigCapital 项目（无客户）
Claude Sonnet 4.6	0.152	5	Twenty CRM 部分完成；BC 客户实体错误
DeepSeek V4 Pro	0.152	5	Twenty CRM 部分完成；BC 客户实体错误
Gemini 3.1 Pro	0.182	6	BC 公司客户正确；里程碑 2 发票正确
GPT-5.4 High	0.212	7	BC 公司客户正确；发票细节错误
Claude Opus 4.6	0.242	8	Twenty CRM 部分完成；BC 个人实体（类型错误）
Qwen 3.6 Plus	0.303	10	BC 公司客户；里程碑 2 发票与付款正确

SaaS 环境中尤其如此，因为异步后端处理、客户端缓存和 UI 状态不同步会使动作执行与动作实际生效之间出现不可忽略的差距。

5.4 单次运行足以评估智能体吗？

智能体评测通常报告单次运行（pass@1）性能。我们的结果表明，这种做法可能严重误导，因为同一模型在同一任务的独立运行之间会出现巨大分数方差。

案例：模型内方差（**bof_155**）。我们在 HR 申诉 workflow（**bof_155**）上对 Claude Sonnet 4.6 运行了三次，结果差异惊人：

运行	分数	所得/28	通过的检查
r2	0.000	0	无
r0	0.214	6	申诉类型、申诉记录、员工调动、培训项目
r1	0.679	19	全部 ERPNext 检查、两笔费用以及全部三项 Twenty CRM 任务

运行 2 得到零分，完全没有有效推进任务；运行 0 完成 ERPNext 阶段后陷入停滞；而运行 1 则完成 ERPNext，正确记录两笔费用，并创建全部三项 Twenty CRM 后续任务（获得 28 分中的 19 分），只在 Pretix 阶段失败。与多数智能体不同，它没有让这次失败阻塞其他彼此独立的下游操作。0.000–0.679 的范围体现的是定性差别，而不只是定量差别：它横跨“完全失败”与“大部分完成”。由于每次运行前 SaaS 环境都被重置到相同初始状态，这种方差不能归因于环境随机性。

案例：跨模型与跨运行分化（**bof_023**）。费用报销任务进一步说明了方差如何同时出现在模型之间与同一模型的不同运行之间：

模型	运行 0	运行 1	运行 2
Claude Opus 4.6	0.80	—	—
Qwen 3.6 Plus	0.80	0.70	0.10
Gemini 3.1 Pro	0.10	0.10	0.70
Doubao Seed 2.0 Pro	0.10	0.10	0.10
Kimi K2.5	0.70	—	—
GPT-5.4 High	0.30	—	—

其中呈现出几种模式。第一，同一模型可能在近乎成功与近乎失败之间剧烈摆动：Qwen 在运行 0 得到 0.80，运行 2 却降至 0.10；Gemini 则呈相反模式，运行 0–1 均为 0.10，运行 2 升至 0.70。第二，一些模型始终得分很低（Doubao 三次均为 0.10），更像确定性失效模式，而非随机方差。第三，相同的单次最佳分数（Opus 与 Qwen 都达到 0.80）可能掩盖完全不同的可靠性特征：Qwen 的方差表明其峰值表现无法稳定复现。

分岔点与路径依赖。这种方差的根源在于长程 SaaS 工作流具有路径依赖。在关键决策点——首先进入哪个应用、如何解释含义模糊的表单字段、失败后是重试还是继续——智能体随机采样中的微小差异会导向根本不同的执行轨迹。一旦智能体走上次优路径，例如在不熟悉的 UI 元素上用 50 步反复尝试无效方法（**bof_155** 的 Training Event 案例即如此），剩余步数预算可能不足以恢复。这种分岔动态解释了为何复杂多应用任务的方差最高，而较简单单应用操作的方差最低：决策点越多，早期分化经累积后演变成截然不同结果的机会就越多。

这些发现进一步支持第 4.2 节的 pass@k 分析，并对评测与部署都有实际启示。就评测而言，只报告 pass@1 会错误刻画智能体能力：高方差模型可能仅因选择了不同运行，就显得很强或很弱；pass@k 与分数方差等多次运行指标能够提供丰富得多的信息。就部署而言，真实 SaaS 任务上观察到的高方差说明，生产 CUA 系统将受益于重试机制、集成策略或基于检查点的恢复，以减轻不利的早期轨迹决策所造成的影响。

6 结论

我们提出了 SaaS-Bench，一个用于在真实且可部署的 SaaS 环境中评估 CUA 的基准。SaaS-Bench 建立在六个专业领域的 23 个真实 SaaS 系统之上，包含 106 项需要跨应用协调、多模态理解和长程执行的任务。评测表明，当前智能体在真实 SaaS 工作流中仍然表现困难，尤其是在规划、状态跟踪和错误恢复方面。对于标题提出的问题——CUA 能否利用真实 SaaS 完成专业工作流？——答案是：尚且不能。即使最强模型，端到端完成的完整工作流也不足 4%；而且所有模型都随着任务推进呈现单调性能衰减，说明持续的长程执行与跨应用协调仍是根本性的开放挑战。我们希望 SaaS-Bench 能够推动未来研究迈向更可靠、真正可实际部署的 CUA。

参考文献

- Anthropic. Introducing computer use, a new claude 3.5 sonnet, and claude 3.5 haiku. <https://www.anthropic.com/news/3-5-models-and-computer-use>, October 2024.
- Anthropic. Claude opus 4.7. <https://www.anthropic.com/news/claude-opus-4-7>, April 2026a.
- Anthropic. Claude opus 4.6. <https://www.anthropic.com/news/claude-opus-4-6>, February 2026b.
- Anthropic. Claude sonnet 4.6. <https://www.anthropic.com/news/claude-sonnet-4-6>, February 2026c.
- Léo Boisvert, Megh Thakkar, Maxime Gasse, Massimo Caccia, Thibault Le Sellier De Chezelles, Quentin Cappart, Nicolas Chapados, Alexandre Lacoste, and Alexandre Drouin. WorkArena++: Towards compositional planning and reasoning-based common knowledge work tasks. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024. URL <https://arxiv.org/abs/2407.05291>.
- ByteDance Seed. Seed2.0 model card: Towards intelligence frontier for real-world complexity. <https://l3-static.bytednsdoc.com/obj/eden-cn/lapzild-tss/ljhwZthlaukjlkulzlp/seed2/0214/Seed2.0%20Model%20Card.pdf>, February 2026.
- Chrystal R. China. What is software as a service (saas)? <http://www.ibm.com/think/topics/saas>, 2025.
- DeepSeek AI. Deepseek-v4: Towards highly efficient million-token context intelligence. https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek_V4.pdf, April 2026.
- Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2Web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. WorkArena: How capable are web agents at solving common knowledge work tasks? *arXiv preprint arXiv:2403.07718*, 2024. URL <https://arxiv.org/abs/2403.07718>.
- Gartner. Gartner forecasts worldwide public cloud end-user spending to total \$723 billion in 2025. <https://www.gartner.com/en/newsroom/press-releases/2024-11-19-gartner-forecasts-worldwide-public-cloud-end-user-spending-to-total-723-billion-dollars-in-2025>, November 2024.
- Google DeepMind. Gemini 3.1 pro model card. <https://deepmind.google/models/model-cards/gemini-3-1-pro>, February 2026a.
- Google DeepMind. Gemini 3.5 flash model card. <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-5-Flash-Model-Card.pdf>, May 2026b.
- Boyu Gou et al. Mind2Web 2: Evaluating agentic search with Agent-as-a-Judge. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2025. URL <https://arxiv.org/abs/2506.21506>.
- Shibo Hao et al. CocoaBench: Evaluating unified digital agents in the wild. *arXiv preprint arXiv:2604.11201*, 2026. URL <https://arxiv.org/abs/2604.11201>.
- Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. Webvoyager: Building an end-to-end web agent with large multimodal models, 2024. URL <https://arxiv.org/abs/2401.13919>.
- Kimi Team. Kimi k2.5: Visual agentic intelligence, 2026a. URL <https://arxiv.org/abs/2602.02276>.

- Kimi Team. Kimi k2.6: Advancing open-source coding. <https://www.kimi.com/blog/kimi-k2-6>, April 2026b.
- Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. VisualWebArena: Evaluating multimodal agents on realistic visual web tasks. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. URL <https://arxiv.org/abs/2401.13649>.
- Evan Zheran Liu, Kelvin Guu, Panupong Pasupat, Tianlin Shi, and Percy Liang. Reinforcement learning on web interfaces using workflow-guided exploration. In *Proceedings of the Sixth International Conference on Learning Representations (ICLR)*, 2018.
- MiniMax. Minimax m2.7: Early echoes of self-evolution. <https://www.minimax.io/news/minimax-m27-en>, March 2026.
- Magnus Müller and Gregor Pijon. Browser-use: Make websites accessible for AI agents. <https://github.com/browser-use/browser-use>, 2024. Open-source framework.
- OpenAI. Computer-using agent. <https://openai.com/index/computer-using-agent/>, January 2025.
- OpenAI. Introducing gpt-5.4. <https://openai.com/index/introducing-gpt-5-4>, March 2026a.
- OpenAI. Introducing gpt-5.5. <https://openai.com/index/introducing-gpt-5-5/>, April 2026b.
- Yujia Qin et al. Ui-tars: Pioneering automated gui interaction with native agents, 2025. URL <https://arxiv.org/abs/2501.12326>.
- Qwen Team. Qwen3.6-Plus: Towards real world agents, April 2026. URL <https://qwen.ai/blog?id=qwen3.6>.
- Christopher Rawles, Sarah Clinckemaiellie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillcrap, and Oriana Riva. Androidworld: A dynamic benchmarking environment for autonomous agents, 2025. URL <https://arxiv.org/abs/2405.14573>.
- Xinyuan Wang et al. Opencua: Open foundations for computer-use agents, 2025. URL <https://arxiv.org/abs/2508.09123>.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multi-modal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL <https://arxiv.org/abs/2404.07972>.
- Frank F Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Z Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, et al. Theagentcompany: benchmarking llm agents on consequential real world tasks. *arXiv preprint arXiv:2412.14161*, 2024.
- Tianci Xue, Weijian Qi, Tianneng Shi, Chan Hee Song, Boyu Gou, Dawn Song, Huan Sun, and Yu Su. An illusion of progress? Assessing the current state of web agents. In *Conference on Language Modeling (COLM)*, 2025. URL <https://arxiv.org/abs/2504.01382>.
- Zhipu. Glm-5.1: Towards long-horizon tasks. <https://z.ai/blog/glm-5.1>, April 2026.
- Shuyan Zhou. WebArena-Infinity: Generating browser environments with verifiable tasks at scale. *Technical Blog Post*, 2026. URL <https://webarena.dev/webarena-infinity/>.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.